

CAMBA CPS-UAV at the SBFT Tool Competition 2024

Marco De Liso
University of Bern
Switzerland

marco.deliso@students.unibe.ch

Zhi Wen Soi
University of Bern
Switzerland

zhi.soi@students.unibe.ch

ABSTRACT

In the past decade, Unmanned Aerial Vehicles (UAVs) have surged in popularity, evolving from niche applications to indispensable tools across diverse industries. This escalating interest is underscored by the pursuit of more reliable autonomous systems, pushing UAVs into crucial roles. Therefore, the aim will be to produce test case generation algorithms to identify software flaws related to self-driving drones. In this work, we will introduce Cost-Aware Mutation-Based (CAMBA), a test case generation algorithm for UAVs that produces a smart obstacle-placement system to make it hard for the drone to complete the mission safely, leading to unsafe flights or even crashes. We will show that our algorithm can reach up to 17% globally dangerous flight scenarios and produce results in 5/6 of the missions. Furthermore, CAMBA reached third place in the global ranking of the competition and first place among the 2-obstacle test generator tools. The implementation can be found at CAMBA.

ACM Reference Format:

Marco De Liso and Zhi Wen Soi. 2024. CAMBA CPS-UAV at the SBFT Tool Competition 2024. In *2024 ACM/IEEE International Workshop on Search-Based and Fuzz Testing (SBFT '24)*, April 14, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3643659.3643935>

1 INTRODUCTION

UAVs, or drones, are autonomous copters fully independent, hence able to complete a mission by themselves. However, real-world testing reveals a lack of resiliency in modern drones. To address this, two testing approaches are employed: field testing and simulation-based testing. Our focus is on simulation-based testing due to its efficiency, cost-effectiveness, and safety in simulating urban environments. This method optimizes the workflow by ensuring stability and reliability through testing before field deployment.

In this context, Search-Based and Fuzz Testing (SBFT) started a competition [3] in collaboration with ICSE24 [2] conference, inviting all participants to develop a test case generation algorithm for the open-access project using the unmanned AERIAL vehicle teST bench (Aerialist) [2, 4] and PX4-Autopilot [5] avoidance system infrastructure. Aerialist is a novel test bench for UAV software that automates all the necessary testing steps, powered by PX4-Autopilot (a widely used open-source UAV firmware). State-of-the-art works such as Khatiri et al. [1] tried to address the issues in autonomous

drones by providing a solution called Surrealist, where they propose a mutation-based algorithm that can identify several problems in the PX4 coupled with Aerialist. However, even if the algorithm detects many bugs in the software used, it is not yet enough to guarantee a fully resilient and robust system. Hence, alternative algorithms are needed to comprehensively test Aerialist and PX4. The main contributions of this work are listed hereafter:

- We propose CAMBA, a cost-aware, mutation-based algorithm that positions the obstacles in the environment by sequentially learning from previous flight logs, to make the drone crash.
- Through simulations, we demonstrate that CAMBA consistently creates challenging conditions for the drone, generating a total of 17% challenging and diverse environments.
- We provide insights into the competition and present the ranking achieved by CAMBA, comparing it with the baseline Surrealist.

2 CAMBA

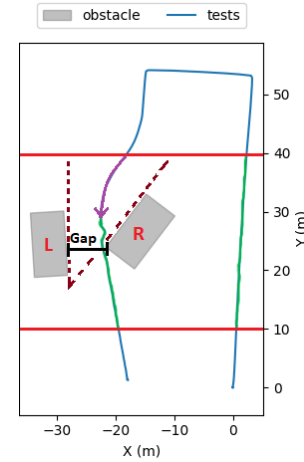


Figure 1: Example simulation.

The algorithm is specialized for missions with two obstacles and the techniques employed are unsuitable for more than two obstacles. We define a graph $G = (B, T)$, where $B \in G$ represents the set of the obstacles $L, R \in B$ placed in G and L and R are the left and right obstacles as shown in Figure 1 and $T \in G$ is the set of (x, y) points of the flight path trajectory of the drone. Furthermore, the algorithm will not consider the whole trajectory path of the drone, instead, it will use a part of it, named *sample trajectory* $T^* \in T$ to avoid outliers. In addition, $d(i, T)$ is the Euclidean distance from the obstacle $i \in B$ to the trajectory T , and $d(L, R)$ is the distance between the two obstacles $L, R \in B$. We also define $d(i, T)$ as the *target distance* which



This work licensed under Creative Commons Attribution-ShareAlike 4.0 International License.

SBFT '24, April 14, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0562-5/24/04

<https://doi.org/10.1145/3643659.3643935>

represents the minimum distance to be achieved to consider the test case as dangerous. The obstacles have a position and size $B(P, S)$, where $P = (x, y, z, r)$ is the position vector of the obstacle that specifies (x, y, z) in meters and the rotation r in degrees, whereas $S = (l, w, h)$ is the size vector defining its length, width, and height in meters as well. Finally, the concept of *budget* is used to define the number of iterations available to the algorithm. We distinguish two main directions for the sake of the competition: *forward* and *backward*. The algorithm has a learning process from previous missions to plan the next steps on the obstacle mutations. We can distinguish two main parts: the *initialization* step and the *iterative learning-based mutations* step.

2.1 Initialization

To accurately place obstacles, the algorithm initially acquires the *original trajectory* by executing the mission without obstacles. This is crucial for positioning obstacles near the drone trajectory within competition boundaries while maintaining an optimal trajectory T . From T , CAMBA extracts T^* and discards the rest of the trajectory to avoid potential BIAS in complex missions. In Figure 1, T^* represents a segment of the drone trajectory within the competition boundaries. Insights into the drone's direction can be gleaned from the sample trajectory T^* ; for instance, the drone is considered to fly forward when x is fixed and y increases or backward when y decreases, as illustrated by the violet arrow in Figure 1.

We use the Halton distribution to partially randomize initial obstacle properties S and P . The values are drawn from a specified range within competition boundaries, ensuring alignment with mission feasibility. Key constraints, such as $d(L, R) > d(L, T^*)$, are imposed to maintain an appropriate distance between obstacles and prevent overlap with T^* . The algorithm aims to preserve and manipulate a V-shape during mutations, defined by the angle formed by the obstacles, dependent on the drone's incoming direction. In Figure 1, the brown dashed lines illustrate this V-shape, which orients towards the drone direction when flying backward (violet arrow). The algorithm monitors obstacles L and R to ensure mutations reflect their placement relative to unmutated obstacles.

2.2 Iterative Learning-based Mutations

Post-initialization, the algorithm enters an iterative process. For iterations n and $n - 1$, the algorithm utilizes the trajectory record based on obstacle placement at $n - 1$ to identify the current scenario n . Three scenarios are distinguished: *gap mutation*, *left mutation*, and *right mutation*. Gap mutation occurs when the trajectory path from the previous iteration passes through the gap between obstacles (Figure 1). If the target distance is not reached, both obstacles are mutated to slightly shift towards each other. In the left mutation scenario, the drone navigates to the left of obstacle L , and the mutation focuses on obstacle L to guide the drone back into the gap. Parameters w , x , and r are adjusted to increase the gap and V-shape angle. The right mutation is similar but for obstacle R . The algorithm considers the drone's incoming direction to ensure logical mutations. This iteration repeats until the target distance is achieved, at which point the algorithm resets parameters and restarts the process.

3 EVALUATION

In our experiment, two obstacles were positioned within a predefined area, highlighted in red in Figure 1, with $x \in [-40m, 30m]$ and $y \in [10m, 40m]$. Additionally, CAMBA underwent evaluation in six distinct missions, each assigned a budget of 200. Subsequently, each valid test case underwent five replications to uncover non-deterministic drone behaviors. To ensure comparability, we computed the Dynamic Time Warping (DTW). Test cases with a DTW value below the 75 threshold were deemed too similar to other evaluated cases. The scores were assigned based on the minimum distance achieved, with the crash giving the highest score. Unsafe conditions were identified when the drone failed to uphold a minimum safe distance of 1.5m from nearby obstacles, while a crash involved the drone colliding with any obstacle.

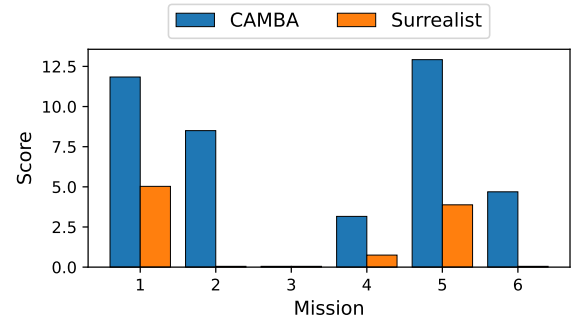


Figure 2: Comparison between CAMBA and Surrealist.

Metric	1	2	3	4	5	6	Average
Crash Rate	5%	0%	0%	4.54%	5.56%	0%	2.52%
Unsafe Rate	20%	20%	0%	0%	16.67%	30%	14.44%
DTW	3516	2984	362	2230	8021	4866	3663

Table 1: Evaluation of the six different missions.

All generated test cases had DTW scores above 75 as depicted in Table 1, showcasing CAMBA's ability to produce diverse test cases. Figure 2 demonstrates CAMBA's superior performance over Surrealist in multiple missions, except mission 3, where both failed to yield results. Table 1 reveals that CAMBA maintained an average crash rate of 2.52% and an average unsafe rate of 14.44%.

Overall, CAMBA successfully generated failing test cases for 5 out of the 6 missions, securing third place in the global ranking of the competition. It excelled as the top performer, showcasing its capability to generate diverse test cases using only two obstacles, considered the main challenge in the competition.

REFERENCES

- [1] Sajad Khatiri, Sebastiano Panichella, and Paolo Tonella. 2023. Simulation-based test case generation for UAVs in the neighborhood of real flights. In *2023 16th IEEE International Conference on Software Testing, Verification and Validation (ICST)*.
- [2] Sajad Khatiri, Sebastiano Panichella, and Paolo Tonella. 2024. Simulation-based Testing of Unmanned Aerial Vehicles with Aerialist. In *International Conference on Software Engineering (ICSE)*.
- [3] Sajad Khatiri, Prasun Saurabh, Timothy Zimmermann, Charith Munasinghe, Christian Birchler, and Sebastiano Panichella. 2024. SBFT Tool Competition 2024 - CPS-UAV Test Case Generation Track. In *IEEE/ACM International Workshop on Search-Based and Fuzz Testing, SBFT@ICSE 2024*.
- [4] Sajad Khatiri et al. 2024. *Aerialist*. <https://github.com/skhatiri/Aerialist.git>
- [5] Lorenz Meier et al. 2024. *PX4-Autopilot*. <https://github.com/PX4/PX4-Autopilot.git>